PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Зачем нужен этот урок

Содержание

- Что такое CSV-файл
- Мини-набор pandas
- Краткая памятка

Перед анализом данных важно убедиться, что программа видит таблицу именно так, как задумано: отдельные строки не склеились, названия столбцов распознаны, а числа не превратились в текст. Такая первичная проверка помогает обнаружить ошибки чтения до того, как они повлияют на результаты вычислений.
В предыдущих практикумах таблица уже рассматривалась как датасет: одна строка описывает объект, столбцы хранят данные об объекте. Теперь добавляется более технический навык: открыть CSV-файл, проверить, что он прочитан правильно, найти нужные записи и получить первые простые показатели.
Главный результат занятия: программа первичной обработки таблицы. Она открывает CSV, выводит размер, названия столбцов, считает записи по условиям и показывает, как читать файл с другим разделителем и кодировкой.
Эти действия образуют базовую последовательность работы: сначала загрузить данные, затем проверить структуру таблицы и только после этого выполнять фильтрацию и подсчёты.

Что такое CSV-файл

CSV — это текстовый формат хранения таблицы. Значения в строке разделяются специальным символом: чаще всего запятой, но иногда точкой с запятой. Pandas читает такой текст и превращает его в `DataFrame` — таблицу внутри Python.
Первая строка CSV-файла обычно содержит заголовки столбцов, а следующие строки — данные отдельных объектов. Все строки должны быть разделены переносами, а значения внутри строки — одним и тем же разделителем.

```
student_id,age,country,sleep_hours,stress_level
1,16,Japan,7.2,6
2,17,Brazil,6.1,8
```

В этом примере первая строка задаёт пять названий столбцов. После чтения файла pandas создаст две записи: одну для объекта со значением `student_id`, равным `1`, и вторую для объекта со значением `student_id`, равным `2`.
Расширение `.csv` само по себе не гарантирует, что внутри файла используется запятая. Если данные разделены точкой с запятой, этот символ необходимо явно указать через параметр `sep`. Иначе pandas может прочитать всю строку как один столбец.
Если файл прочитан неверно, дальнейшие расчёты тоже будут неверными. Поэтому перед анализом нужно вывести первые строки, размер таблицы, названия столбцов и типы данных. Например, один столбец вместо нескольких может указывать на неверно выбранный разделитель. Если числовой столбец распознан как текстовый, стоит проверить его значения: среди них могут находиться лишние символы или записи другого формата.

> **Важно:**
>
> кодировка и разделитель решают разные задачи. `sep` определяет границы между значениями, а `encoding` сообщает, как преобразовать байты файла в читаемые символы.

Мини-набор pandas

Обычно библиотеку pandas подключают один раз в начале программы. Сокращение `pd` используется в следующих командах для обращения к функциям библиотеки, а переменная `df` обычно хранит полученный `DataFrame`.

Команда	Действие
`import pandas as pd`	подключить pandas
`pd.read_csv("file.csv")`	открыть CSV-файл
`df.head()`	вывести первые строки
`df.tail()`	вывести последние строки
`df.sample(5)`	вывести случайные строки

Команда	Действие
`len(df)`	определить количество записей
`df.shape`	посмотреть строки и столбцы
`df.columns`	вывести названия столбцов
`df.info()`	посмотреть типы и заполненность
`df[df["country"] == "Japan"]`	найти строки по условию
`pd.read_csv("file.csv", sep=";", encoding="cp1251")`	прочитать файл с другим разделителем и кодировкой

Результат `pd.read_csv` обычно сохраняют в переменную. После этого команды `head`, `shape`, `columns` и другие применяются к одной и той же загруженной таблице.

Команды `df.head()` и `df.tail()` по умолчанию показывают по пять строк. Они позволяют проверить разные части таблицы: начало удобно для проверки заголовков и первых записей, а конец — для поиска лишних строк или неожиданного завершения данных.

Команда `df.sample(5)` выбирает пять случайных строк, поэтому при повторных запусках результат может меняться. Если в таблице меньше пяти строк, запрос пяти случайных строк без дополнительных параметров выполнить не получится.

Значения `len(df)` и `df.shape` связаны, но не совпадают по форме результата. `len(df)` возвращает только количество строк, а `df.shape` — пару значений: сначала количество строк, затем количество столбцов. Для таблицы из двух строк и пяти столбцов `df.shape` даст результат `(2, 5)`.

`df.columns` выводит имена всех столбцов. Это полезно перед фильтрацией: имя в условии должно точно совпадать с названием столбца, включая регистр, пробелы и символы подчёркивания.

`df.info()` помогает проверить не только типы данных, но и количество непустых значений в каждом столбце. Если число заполненных значений меньше общего количества строк, значит, в столбце есть пропуски.

Выражение `df["country"] == "Japan"` проверяет условие отдельно для каждой строки. Затем `df[...]` оставляет только те записи, для которых условие истинно. В данном примере будет выбрана строка, где в столбце `country` указано значение `Japan`.

Тип значения в условии имеет значение: текст записывается в кавычках, а число — без кавычек. Например, значение `"16"` является строкой, а `16` — числом; для числового столбца `age` обычно требуется сравнение с числом.

При чтении файла с параметрами `sep=";"` и `encoding="cp1251"` pandas использует точку с запятой как границу между значениями и кодировку `cp1251` для распознавания текста. Эти параметры следует задавать по фактическому устройству файла, а не по его названию.

Краткая памятка

- `df.columns` показывает названия столбцов, а не автоматически «признаки». Некоторые столбцы могут быть идентификаторами или служебными данными и не использоваться как признаки в конкретной задаче.
- Для одного условия используется `df[df["столбец"] == значение]`. Результатом будет новый `DataFrame` только с подходящими строками; исходная таблица от такой команды не изменяется.
- Для двух условий каждое условие берётся в скобки, между ними ставится `&`. Скобки обязательны, потому что pandas должен сначала вычислить каждое сравнение отдельно, а затем объединить результаты.
- Знак `&` означает одновременное выполнение двух условий. Если нужно выбрать строки, удовлетворяющие хотя бы одному из условий, используется знак `|`, а каждое условие также заключается в скобки.
- `sep` задаёт разделитель значений в CSV. Если после чтения вместо ожидаемых нескольких столбцов получился один, сначала следует проверить именно этот параметр.
- `encoding` задаёт способ чтения текста в файле. Неверная кодировка может привести к ошибке чтения или неправильному отображению символов.
- Перед подсчётом записей по условию полезно проверить `df.columns` и несколько строк таблицы: это снижает риск ошибки в названии столбца или в записанном значении.
- Лишний текст в `print` может помешать автоматической проверке. Если требуется вывести только число или список столбцов, подписи и дополнительные пояснения добавлять не следует.
- Мини-проверка: для таблицы размером 20 строк и 6 столбцов `len(df)` вернёт 20, а `df.shape` — `(20, 6)`.

Материалы к занятию

- lesson_1_15_csv_10_11_spo_reworked.pptx (210.3 КБ)

Файлы практикума

Содержание

- Практическая часть
- Материалы к практике

В практикуме используются два CSV-файла с разными параметрами хранения данных. Это позволяет отработать не только чтение таблицы, но и правильное указание разделителя и кодировки.

- `student_lifestyle_1_15_10_11_spo.csv` — основной CSV в UTF-8, 60 строк и 12 столбцов. Каждая строка соответствует одной записи, а каждый столбец — отдельному признаку.
- `student_lifestyle_ru_semicolon_cp1251_1_15.csv` — пример CSV с русскими названиями столбцов, разделителем `";"` и кодировкой `cp1251`. При чтении такого файла параметры разделителя и кодировки необходимо указать явно.
- Файлы учебные и не содержат реальных персональных данных. Их содержимое предназначено только для выполнения заданий практикума.

Расширение `.csv` не гарантирует, что все файлы читаются одинаково: в них могут использоваться разные разделители и кодировки. Если указать неверные параметры, столбцы могут определиться неправильно или при чтении появится ошибка декодирования.

Практическая часть

В стартовом ноутбуке часть кода уже подготовлена. В основных заданиях нужно дописывать отдельные строки и получать точный вывод. Для платформенной проверки важно, чтобы программа выводила только требуемое число или строку без дополнительных пояснений. Например, если требуется вывести количество подходящих записей, не следует добавлять перед числом подпись. Вывод `15` и вывод `Количество записей: 15` воспринимаются системой проверки как разные результаты.

```
import pandas as pd

df = pd.read_csv("student_lifestyle_1_15_10_11_spo.csv")
print(df.shape[0], df.shape[1])
```

Функция `pd.read_csv()` читает CSV-файл и создаёт объект `DataFrame`. Переменная `df` хранит полученную таблицу, а атрибут `shape` содержит её размеры: сначала количество строк, затем количество столбцов. Поэтому `df.shape[0]` равно числу записей, а `df.shape[1]` — числу признаков.
Для основного файла приведённый код выведет:

```
60 12
```

Частая ошибка — менять индексы местами и считать, что `shape[0]` обозначает столбцы. В pandas индекс 0 относится к строкам, а индекс 1 — к столбцам.
После открытия файла можно считать записи по условиям: страна, использование ИИ для учёбы, высокий стресс, короткий сон. Для этого условие применяется к соответствующему столбцу, а затем подсчитывается количество строк, для которых оно выполняется. При сочетании нескольких условий важно учитывать, должны ли они выполняться одновременно или достаточно выполнения одного из них.
Затем отдельно рассматривается файл с разделителем `";"` и кодировкой `cp1251`. Его можно открыть, передав необходимые параметры функции чтения:

```
df_ru = pd.read_csv("student_lifestyle_ru_semicolon_cp1251_1_15.csv", sep=";", encoding="cp1251")
```

Параметр `sep=";"` сообщает pandas, что значения разделены точкой с запятой, а `encoding="cp1251"` задаёт кодировку текста. Эти параметры относятся к формату файла и не изменяют сами данные.
Обратите внимание: дополнительные вызовы `print()` удобны при отладке, но перед платформенной проверкой их следует удалить или отключить, если задание требует единственный точный результат.
Для самопроверки можно определить, что обозначают значения `df.shape[0]` и `df.shape[1]`, а также объяснить, зачем при чтении второго файла нужны параметры `sep` и `encoding`.

Материалы к практике

- lesson_1_15_csv_10_11_spo_start.ipynb (3.3 КБ)
- student_lifestyle_1_15_10_11_spo.csv (3.0 КБ)
- student_lifestyle_ru_semicolon_cp1251_1_15.csv (609 Б)